

# QA Prompt Library

## 10 Reusable Claude Prompts Every QA Engineer Should Have Bookmarked

*Day 14 of 20 — Teaching QA Engineers to Master Claude AI*

This document contains 10 battle-tested reusable prompts designed for modern QA teams using Claude AI and other LLMs for software testing workflows.

These prompts are structured to improve:

- Consistency across QA teams
- Test coverage quality
- AI-assisted validation workflows
- Automation productivity
- Requirement analysis accuracy

Each prompt is parameterized and reusable across domains, products, and project types.

### 01 — Test Case Generator

You are a senior QA engineer. Given the following user story, generate a structured test case suite including: positive flows, negative flows, boundary conditions, and edge cases. Output as a table with columns: Test ID | Description | Steps | Test Data | Expected Result.

User Story: [PASTE HERE]

### 02 — BDD Scenario Generator

Convert the following requirement into Gherkin BDD scenarios. Include: happy path, at least 2 negative scenarios, and 1 edge case. Use proper Given/When/Then format. Domain: [DOMAIN].

Requirement: [PASTE HERE]

### 03 — API Negative Test Suite

You are a QA engineer testing a REST API endpoint. Generate a comprehensive negative test input suite covering: missing required fields, invalid data types, boundary values, malformed JSON, authentication failures, and concurrent request edge cases.

Endpoint: [METHOD + PATH]

Request Schema: [PASTE SCHEMA]

## 04 — Bug Report Formatter

Transform the following rough bug observation into a professional bug report with: Title, Environment, Steps to Reproduce, Actual Result, Expected Result, Severity, and Root Cause Hypothesis.

Observation: [PASTE HERE]

## 05 — Test Plan Gap Reviewer

You are a QA architect. Review this test plan and identify: missing coverage areas, untested risk zones, ambiguous acceptance criteria, and scenarios likely to cause production issues.

Test Plan: [PASTE HERE]

## 06 — Playwright Script Generator

Generate a Playwright (TypeScript) test script for the following manual test steps. Handle: dynamic element waits, assertions on state change, and add a clear test description block.

Manual Steps: [PASTE HERE]

Application Context: [BRIEF DESCRIPTION]

## 07 — LLM Output Validator

You are an AI QA evaluator. Evaluate the following LLM response against the expected criteria. Score on: factual accuracy (1-5), hallucination risk (1-5), completeness (1-5), and tone alignment (1-5). Provide a brief justification for each score.

LLM Response: [PASTE HERE]  
Expected Criteria: [PASTE HERE]

## 08 — Prompt Injection Probe Set

Generate 10 adversarial prompt injection test inputs designed to test the robustness of an AI system handling user input. Include: jailbreak attempts, instruction override patterns, role confusion attacks, and data exfiltration probes.

Domain: [SYSTEM TYPE]

## 09 — Requirements Ambiguity Extractor

Review the following requirements document and flag every ambiguous statement, missing constraint, undefined term, and untestable acceptance criterion. Output as a numbered list with suggested clarification questions for each.

Requirements: [PASTE HERE]

## 10 — Sprint Retrospective QA Analyzer

Analyze the following list of bugs found in this sprint. Identify patterns, classify by defect origin (requirements/design/code/test), and recommend 3 process improvements to prevent recurrence.

Bug List: [PASTE HERE]

### How to Use This Library Effectively

1. Standardize prompts across the QA team.
2. Create prompt templates inside Confluence/Notion.
3. Review AI-generated outputs before production usage.
4. Continuously refine prompts based on sprint learnings.
5. Treat prompts as reusable QA assets.

AI will not replace QA engineers.

But QA engineers who effectively use AI will outperform teams that do not.

*Created for LinkedIn Series: Teaching QA Engineers to Master Claude AI*